Lecture Notes: TwinCAT 3

Handling Different TwinCAT 3 Versions

March 17, 2026

Contents

1 The Challenge of Version Management **2**

2 TwinCAT Installation Components **2**

3 The Problem: Automatic Updates **3**

4 How to "Freeze" a Project for Long-Term Maintenance **5**

4.1 A. Freezing Library Placeholders 5

4.2 B. Pinning the Compiler and Project Version 5

5 Professional Workflow and Build Integrity **6**

6 Industry Context **9**

Quick Reference Card **9**

1 The Challenge of Version Management

In industrial automation, machines often run for years or decades. As a developer, you will eventually manage several PLCs, each potentially running a different version of **TwinCAT**. If the version on your development machine (**XAE**) does not match the version on the target machine (**XAR**), you may encounter issues when attempting to perform maintenance or minor code changes.

Scenario	Risk Level	Potential Issues
XAE = XAR version	Low	None expected
XAE newer than XAR	Medium	Library mismatch, compiler differences
XAE older than XAR	High	Missing features, compatibility errors
Unfrozen project	Critical	Unpredictable binary changes

Table 1: Version Mismatch Risk Assessment

Warning: Opening an old project with a newer TwinCAT version can silently change the compiled binary, even if you don't modify any code. This can cause unexpected behavior on machines that have been running reliably for years.

2 TwinCAT Installation Components

- **XAE (Engineering):** The full development environment bundled with Visual Studio, the XAR, libraries, and compilers.
- **XAR (Runtime):** The component that executes the code on the PLC.
- **Remote Manager:** A specific download from Beckhoff that includes only the **libraries and compilers** for a particular TwinCAT version.
  - **Installation Rule:** You can install newer XAE versions alongside older ones, but you cannot install an older XAE on a machine that already has a newer one.
  - **The Solution:** Use the **Remote Manager** to install older compilers and libraries on a system that already has a newer XAE installed.

Component	Contains	When to Use
XAE Full	VS Integration + XAR + Libs + Compilers	Primary development machine
XAR Only	Runtime execution environment	Production PLCs
Remote Manager	Libs + Compilers (no VS)	Add old versions to new XAE

Table 2: TwinCAT Installation Component Overview

Tip: Download and archive Remote Manager installers for every TwinCAT version used in your production machines. Store them in a shared network location or version control system for long-term accessibility.

Development PC		Target PLC
XAE 4026.0 (Latest) + Remote Manager 4024.12 + Remote Manager 4022.31	→	XAR 4024.12

Figure 1: Multi-Version Development Setup

3 The Problem: Automatic Updates

By default, when you open a TwinCAT project, the IDE automatically selects the **latest version** of the libraries and the **newest version** of the IEC compiler available on your machine.

- **Library Star (*):** In the library manager, a star symbol indicates the project is using "whatever the latest version is".
- **Online Change Warnings:** Even if you haven't changed a single line of code, opening an old project on a machine with newer TwinCAT versions can trigger an "Application change" message. This is because the binary executable differs when compiled with newer libraries or compilers.

Library Placeholder Definition

```
# ====================================================
# LIBRARY PLACEHOLDER CONCEPTS
# ====================================================
# In TwinCAT, library references can be either:
# 1. DYNAMIC (using placeholder with *)
# 2. FROZEN (using specific version number)

# ====================================================
# DYNAMIC REFERENCE (DEFAULT - DANGEROUS FOR MAINTENANCE)
# ====================================================
# The asterisk (*) means "use the latest installed version"
# This is convenient during development but problematic for
# long-term maintenance

Library Placeholder:
    Name:       Tc2_Standard
    Version:    * (Latest)              # <-- DYNAMIC
    Resolution: Tc2_Standard, 3.4.1.0   # Resolves to newest on THIS
   machine

# Problem: On another machine with different TwinCAT version,
# this might resolve to:
#    Resolution: Tc2_Standard, 3.3.2.0   # Different version!

# ====================================================
# FROZEN REFERENCE (RECOMMENDED FOR PRODUCTION)
# ====================================================
# Specifying exact version ensures reproducible builds
# across all development machines and over time

Library Placeholder:
    Name:       Tc2_Standard
    Version:    3.3.2.0                 # <-- FROZEN
    Resolution: Tc2_Standard, 3.3.2.0   # Always this exact version
```

```

Library Placeholder:
    Name:       Tc2_System
    Version:    3.4.3.0                 # <-- FROZEN
    Resolution: Tc2_System, 3.4.3.0     # Always this exact version

Library Placeholder:
    Name:       Tc2_Utilities
    Version:    3.3.45.0                # <-- FROZEN
    Resolution: Tc2_Utilities, 3.3.45.0 # Always this exact version

# ====================================================
# VISUAL COMPARISON IN TWINCAT LIBRARY MANAGER
# ====================================================
#
# BEFORE FREEZING (Dynamic References):
# +-----------------------------------------+
# | References                              |
# |   +-- Tc2_Standard, * (Placeholder)     |  <-- Star = Dynamic
# |   +-- Tc2_System, * (Placeholder)       |  <-- Star = Dynamic
# |   +-- Tc2_Utilities, * (Placeholder)    |  <-- Star = Dynamic
# |   +-- Tc3_Module, * (Placeholder)       |  <-- Star = Dynamic
# +-----------------------------------------+
#
# AFTER FREEZING (Fixed References):
# +-----------------------------------------+
# | References                              |
# |   +-- Tc2_Standard, 3.3.2.0             |  <-- Fixed version
# |   +-- Tc2_System, 3.4.3.0               |  <-- Fixed version
# |   +-- Tc2_Utilities, 3.3.45.0           |  <-- Fixed version
# |   +-- Tc3_Module, 3.1.10.0              |  <-- Fixed version
# +-----------------------------------------+
```

```
# ====================================================
# EXAMPLE: PROJECT FILE DIFFERENCES
# ====================================================
# The .plcproj file stores library references
# Here's what changes when you freeze:

# BEFORE (Dynamic - in .plcproj XML):
<PlaceholderReference Include="Tc2_Standard">
    <DefaultResolution>Tc2_Standard, * (Beckhoff)</DefaultResolution>
    <Namespace>Tc2_Standard</Namespace>
</PlaceholderReference>

# AFTER (Frozen - in .plcproj XML):
<PlaceholderReference Include="Tc2_Standard">
    <DefaultResolution>Tc2_Standard, 3.3.2.0 (Beckhoff)</
   DefaultResolution>
    <Namespace>Tc2_Standard</Namespace>
</PlaceholderReference>

# ====================================================
# DEPENDENCY CHAIN EXAMPLE
# ====================================================
# When you freeze a library, its dependencies should also
# be frozen to ensure complete reproducibility

```

```
Project: MyMachineController
    |
    +-- Tc2_Standard, 3.3.2.0 (frozen)
    |       |
    |       +-- Tc2_System, 3.4.3.0 (auto-frozen as dependency)
    |
    +-- Tc2_Utilities, 3.3.45.0 (frozen)
    |       |
    |       +-- Tc2_System, 3.4.3.0 (same version - OK)
    |       +-- Tc2_Standard, 3.3.2.0 (same version - OK)
    |
    +-- MyCompanyLibrary, 1.2.0.0 (frozen)
            |
            +-- Tc2_Standard, 3.3.2.0 (same version - OK)
```

Note: The asterisk (*) in library placeholders is a convenience feature for active development. However, it should **always** be replaced with specific version numbers before releasing a project to production.

4 How to "Freeze" a Project for Long-Term Maintenance

To ensure you can rebuild an application years later and get the **exact same binary**, you must decouple the project from the latest system updates.

4.1 A. Freezing Library Placeholders

Instead of using the default latest version (*), developers must manually fix libraries to the versions they were originally developed with.

1. Go to the **Placeholders** section in the Library Manager.
2. Select the specific version delivered with that TwinCAT build (e.g., Tc2_Standard version 3.3.2.0).
3. This also forces dependencies (libraries within libraries) to remain at those fixed versions.

4.2 B. Pinning the Compiler and Project Version

You must "pin" the project to a specific TwinCAT build to prevent it from automatically upgrading to the latest installed version.

1. In the project tree, go to **System**.
2. Select **Pin version** to lock the project to the current build (e.g., 4024.12).
3. If a colleague attempts to open this project without that specific version installed, TwinCAT will issue a warning.

Warning: Always test your project after freezing to ensure it compiles correctly with the pinned versions. Some library combinations may have compatibility issues that only become apparent when specific versions are locked together.

Step	Location	Action
1. Freeze Libraries	PLC → References → Placeholders	Select exact versions
2. Pin Compiler	SYSTEM → General	Check "Pin Version"
3. Verify Build	Build → Rebuild Solution	Confirm no warnings
4. Commit Changes	Version Control	Save frozen state

Table 3: Project Freezing Checklist

5 Professional Workflow and Build Integrity

- **Git Integration:** All frozen library and compiler settings should be committed to version control.
- **TwinCAT Functions:** Remember to also track the versions of any **TwinCAT Functions** or supplements (e.g., Vision, OPC-UA, TCP/IP) used, as these also install their own sets of libraries.
- **Final Release:** Before delivering a machine, **freeze everything**—libraries and the compiler—and create a formal release in Git.

Version Control Commands

```
# ======================================================
# GIT WORKFLOW FOR TWINCAT PROJECT RELEASES
# ======================================================
# After freezing all library placeholders and pinning
# the compiler version, commit and tag the release

# ------------------------------------------------------
# Step 1: Check current status
# ------------------------------------------------------
git status

# You should see modified files including:
#   - *.plcproj (library placeholder changes)
#   - *.tsproj (compiler pin settings)
#   - *.sln (solution file)

# ------------------------------------------------------
# Step 2: Stage all changes
# ------------------------------------------------------
git add .

# Or be more selective:
git add *.plcproj
git add *.tsproj
git add *.sln

# ------------------------------------------------------
# Step 3: Commit with descriptive message
# ------------------------------------------------------
git commit -m "Freeze libraries and compiler for Release 1.0

- Pinned TwinCAT version to 4024.12
- Frozen library placeholders:
```

```
  - Tc2_Standard: 3.3.2.0
  - Tc2_System: 3.4.3.0
  - Tc2_Utilities: 3.3.45.0
  - Tc2_MC2: 3.3.42.0
- Tested full rebuild: SUCCESS
- Ready for production deployment"

# -----------------------------------------------------
# Step 4: Create annotated release tag
# -----------------------------------------------------
git tag -a v1.0.0 -m "Release 1.0.0 - Production Ready

Machine: PackagingLine_01
Customer: ACME Manufacturing
TwinCAT Build: 4024.12
Delivery Date: 2024-01-15

Frozen Components:
- All library placeholders locked
- Compiler version pinned
- Full regression test passed"

# -----------------------------------------------------
# Step 5: Push commits and tags to remote
# -----------------------------------------------------
git push origin main
git push origin v1.0.0

# Or push all tags at once:
git push origin --tags

# -----------------------------------------------------
# Step 6: Verify the release
# -----------------------------------------------------
git log --oneline -5
git tag -l

# =====================================================
# ADDITIONAL BEST PRACTICES
# =====================================================

# Create a branch for each major machine/customer
git checkout -b machine/packaging-line-01

# Archive the Remote Manager installer version
# (Store in separate artifact repository or network share)
# Example: TwinCAT_Remote_Manager_4024.12.exe

# Document the development environment
git add ENVIRONMENT.md
git commit -m "Document development environment requirements"
```

```
# =====================================================
# ENVIRONMENT.md - Example Documentation File
# =====================================================
# Include this file in your repository root

cat > ENVIRONMENT.md << 'EOF'
```

```
# Development Environment Requirements

## TwinCAT Version
- **Build:** 4024.12
- **XAE Version:** 3.1.4024.12
- **Visual Studio:** 2019 (16.11.x) or 2022 (17.x)

## Required Remote Managers
If your XAE is newer than 4024.12, install:
- TwinCAT 3.1 Remote Manager Build 4024.12

Download from: https://www.beckhoff.com/download

## Frozen Library Versions
| Library | Version | Notes |
|---------|---------|-------|
| Tc2_Standard | 3.3.2.0 | Core functions |
| Tc2_System | 3.4.3.0 | System utilities |
| Tc2_Utilities | 3.3.45.0 | Extended utilities |
| Tc2_MC2 | 3.3.42.0 | Motion control |
| Tc3_JsonXml | 3.3.18.0 | Data serialization |

## TwinCAT Functions Installed
- TF6100 OPC-UA Server: 3.3.15.0
- TF6310 TCP/IP: 3.3.12.0

## Build Verification
After cloning, perform a full rebuild to verify:
1. Open solution in Visual Studio
2. Build > Rebuild Solution
3. Verify zero errors and zero warnings
EOF
```

```
# ====================================================
# RESTORING A FROZEN PROJECT (Years Later)
# ====================================================

# Clone the repository
git clone https://github.com/company/machine-project.git
cd machine-project

# Checkout the specific release tag
git checkout v1.0.0

# Read the environment requirements
cat ENVIRONMENT.md

# Install required Remote Manager if needed
# (Download from Beckhoff, run installer)

# Open project in TwinCAT
# If version mismatch warning appears:
#   - Install the required Remote Manager
#   - Or adjust project settings (not recommended)

# Rebuild and verify
# The binary should be identical to original release
```

Tip: Create a `ENVIRONMENT.md` or `BUILD_REQUIREMENTS.txt` file in your repository documenting all version requirements. This serves as a quick reference for anyone maintaining the project in the future.

6 Industry Context

Traditional software development uses automated build chains (like Maven or NuGet) to resolve dependencies. Because TwinCAT is based on the **CODESYS platform**, many of these versioning steps are currently manual and require the developer to keep track of configurations themselves. This process is critical because getting it wrong can make software maintenance nearly impossible over a machine's 10-year lifespan.

IT/Web Development	TwinCAT/PLC	Impact
package.json / pom.xml	Manual placeholder config	More error-prone
npm install / mvn build	Manual Remote Manager install	Time-consuming setup
Semantic versioning	Manual version tracking	Requires discipline
CI/CD pipelines	Manual build verification	Less automation

Table 4: Dependency Management: IT vs. Industrial Automation

Machine Lifecycle Timeline		
Year 0	**Year 5**	**Year 10+**
Initial Development	Minor Updates	Major Retrofit
TwinCAT 4024.12	Same version needed	May need old compilers
Libraries frozen	Reproduce exact build	Remote Manager critical

Figure 2: Long-Term Project Maintenance Considerations

Warning: Industrial machines can operate for 15-20 years or more. The TwinCAT version and libraries you use today must be reproducible decades from now. Always maintain archives of installers and documentation.

Quick Reference Card

Version Freezing Checklist

1. **Freeze Libraries:** PLC Project → References → Placeholders → Set specific versions
2. **Pin Compiler:** SYSTEM → General → Pin Version
3. **Document:** Create ENVIRONMENT.md with all version details
4. **Test:** Rebuild solution, verify zero warnings
5. **Commit:** Stage and commit all changes
6. **Tag:** Create annotated Git tag for release
7. **Archive:** Store Remote Manager installer

Key File Locations

Component	Location
Remote Managers	`C:\TwinCAT\3.1\`
Managed Libraries	`...\Components\Plc\Managed Libraries\`
Compilers	`...\Components\Plc\Compilers\`

Git Commands Summary

```
# Stage all changes
git add .

# Commit frozen state
git commit -m "Freeze for Release X.Y"

# Create release tag
git tag -a vX.Y.Z -m "Release description"

# Push everything
git push origin main --tags
```

Version Compatibility Matrix

XAE Version	Can Open Older	Can Target Older XAR	Needs Remote Mgr
4026.x	Yes	Yes (if RM installed)	For older builds
4024.x	Yes	Yes (native)	For 4022 and older
4022.x	Yes	Yes (native)	For 4020 and older

Last Updated: March 17, 2026